

cl-freelock api reference

Andrew D. France

May 22, 2026

Contents

1 Getting Started: Installation

Once available on quicklisp, you will be able to simply run:

```
(ql:quickload :cl-freelock)
```

For now, clone the repository into your Quicklisp local-projects directory:

```
cd ~/quicklisp/local-projects/  
git clone https://github.com/ItsMeForLua/cl-freelock.git
```

Then, load the system in your REPL:

```
(ql:quickload :cl-freelock)
```

2 Basic Usage

In this section, we will go over the basic usage of producer-consumer with unbounded queue, bounded buffers, SPSC, error handling, and atomic operations.

2.1 Producer-Consumer with Unbounded Queue

```
(defparameter *work-queue* (make-queue))  
;; Producer thread  
(defun producer ()  
  (loop for i from 1 to 1000 do  
    (queue-push *work-queue* i)  
    (sleep 0.001)))  
  
;; Consumer thread  
(defun consumer ()  
  (loop  
    (multiple-value-bind (item found)  
      (queue-pop *work-queue*))))
```

```
(queue-pop *work-queue*)  
(if found  
  (format t "Processing: ~A~%" item)  
  (sleep 0.001))))))
```

2.2 Bounded Buffer

```
(defparameter *buffer* (make-bounded-queue 64)) ; Must be power of 2  
  
;; Non-blocking operations  
(when (bounded-queue-push *buffer* "data")  
  (format t "Push succeeded~%"))  
  
(multiple-value-bind (value success)  
  (bounded-queue-pop *buffer*)  
  (when success  
    (format t "Got: ~A~%" value))))
```

2.3 SPSC

```
(defparameter *spsc* (make-spsc-queue 256)) ; Must be power of 2  
  
;; Producer thread only  
(when (spsc-push *spsc* "fast-data")  
  (format t "Pushed successfully~%"))  
  
;; Consumer thread only  
(multiple-value-bind (value success)  
  (spsc-pop *spsc*)  
  (when success  
    (format t "Got: ~A~%" value))))
```

2.4 Error Handling

Operations return multiple values to indicate success/failure rather than signaling conditions:

```
;; Always check the second return value  
(multiple-value-bind (value success)  
  (queue-pop *queue*)  
  (if success  
    (process value)  
    (handle-empty-queue)))
```

2.5 Atomic Operations

Types

2.5.1 ATOMIC-REF

Functions

2.5.2 MAKE-ATOMIC-REF

2.5.3 ATOMIC-REF-VALUE

2.6 Macros

2.6.1 CAS

2.6.2 ATOMIC-INCF

2.7 Utility Functions

2.7.1 MEMORY-BARRIER

2.8 Unbounded Queue

Class

2.8.1 QUEUE

Lock-free unbounded queue supporting multiple producers and consumers.

Functions

2.8.2 MAKE-QUEUE

2.8.3 QUEUE-PUSH

2.8.4 QUEUE-POP

2.8.5 QUEUE-EMPTY-P

2.9 Bounded Queue

Class

2.9.1 BOUNDED-QUEUE

Functions

2.9.2 MAKE-BOUNDED-QUEUE

2.9.3 BOUNDED-QUEUE-CAPACITY

2.9.4 BOUNDED-QUEUE-PUSH

2.9.5 BOUNDED-QUEUE-POP

2.9.6 BOUNDED-QUEUE-EMPTY-P

2.9.7 BOUNDED-QUEUE-FULL-P

Batch Operations

2.9.8 BOUNDED-QUEUE-PUSH-BATCH

2.9.9 BOUNDED-QUEUE-POP-BATCH

2.10 SPSC Queue

Class

2.10.1 SPSC-QUEUE

Functions

2.10.2 MAKE-SPSC-QUEUE

3.1 General Performance Principles

3.1.1 Choose the Right Data Structure

3.1.2 Capacity Selection

3.1.3 Thread Contention

3.2 Optimization Techniques

3.2.1 Batch Operations

3.2.2 Avoid Busy Waiting

3.2.3 Memory Allocation

3.3 Implementation-Specific Notes

3.3.1 SBCL

3.3.2 CLASP

3.4 Common Pitfalls

3.4.1 Wrong capacity

3.4.2 Thread safety

3.4.3 Busy waitings

3.4.4 Memory leaks

3.4.5 False sharing

3.5 Recommended Patterns

3.5.1 Producer-Consumer Pool

3.5.2 Basic Producer-Consumer

3.5.3 Bounded Buffer with Batch Processing

3.5.4 SPSC Pipeline